

Git & GitHub

1. Introduction to Git and GitHub

What is Git?

Git is a **Version Control System (VCS)** used to track changes in code.

It helps developers:

- Save project history
- Track changes
- Go back to previous versions
- Work in teams

Real Life Example

Imagine you are making a project and save files like:

```
project-final  
project-final2  
project-final-final  
project-latest-final
```

This becomes confusing.

Git solves this problem by keeping track of every change automatically.

Definition

Git is a tool used to track and manage changes in files and code.

What is GitHub?

GitHub is an online platform where we store Git repositories on the internet.

It helps:

- Store projects online
- Backup code
- Share projects
- Collaborate with teams

Simple Understanding

Git = Tool

GitHub = Website to store Git projects online

Example:

You create a website on your laptop using Git → Upload it to GitHub → Access it anywhere.

2. Difference Between Git and GitHub

Git	GitHub
Software	Website/Platform
Works locally	Works online
Tracks code changes	Stores repositories
Installed in computer	Used in browser

Example

You write notes in a notebook → **Git**

You upload those notes to Google Drive → **GitHub**

3. What is Version Control System (VCS)?

Version Control System helps manage changes in files.

Example:

Version 1 → Login Page

Version 2 → Added Navbar

Version 3 → Fixed Bugs

Git stores all versions.

You can go back anytime if something breaks.

4. Installing Git

Step 1

Go to:

[Git Official Website](https://git-scm.com/)

Step 2

Download Git according to your operating system.

Step 3

Install normally → Click **Next** → **Next** → **Install**

Step 4

Check installation

Open terminal or command prompt:

```
git --version
```

Example Output:

```
git version 2.50.0
```

This means Git is installed.

5. Git Configuration

Before using Git, we set username and email.

Command

```
git config --global user.name "Your Name"
```

Example:

```
git config --global user.name "Aqib"
```

Email Setup

```
git config --global user.email "youremail@gmail.com"
```

Example:

```
git config --global user.email "aqib@gmail.com"
```

Check Configuration

```
git config --list
```

This shows all settings.

Why is this needed?

Whenever you make changes (commit), Git stores who made the change.

6. Creating a Git Repository

What is Repository?

A repository (repo) is a project folder tracked by Git.

Step 1: Open Project Folder

Example:

```
portfolio-website
```

Step 2: Open Terminal

Right Click → Open terminal

Step 3: Initialize Git

Command:

```
git init
```

What happens?

Git creates hidden tracking files.

Output:

```
Initialized empty Git repository
```

Now Git starts tracking your project.

7. Git Workflow

Git works in 3 stages:

```
Working Directory
  ↓
Staging Area
  ↓
Repository
```

Explanation

1. Working Directory

Where we write code.

Example:

HTML, CSS, JS files.

2. Staging Area

Temporary place where selected files are prepared.

3. Repository

Final saved version of project.

8. Git Status Command

Command

```
git status
```

Purpose

Checks current project status.

Shows:

- Modified files
- New files
- Tracked files

Example:

```
Untracked files:
index.html
style.css
```

Meaning Git found new files.

9. Adding Files to Staging Area

Add Single File

```
git add index.html
```

Add Multiple Files

```
git add .
```

Meaning

Adds files to staging area.

Now Git is ready to save them.

10. Commit Command

What is Commit?

Commit means saving project changes permanently.

Think of it like:

Save Point in Game

Command

```
git commit -m "message"
```

Example:

```
git commit -m "Added homepage"
```

Another Example:

```
git commit -m "Created navbar"
```

Why Message?

Helps understand what changes were made.

Good Message:

```
Added login page  
Fixed navbar issue  
Updated CSS styling
```

Bad Message:

```
abc  
done  
update
```

11. Git Log

Command

```
git log
```

Purpose

Shows project history.

Example Output:

```
commit 123456  
Author: Aqib  
Message: Added navbar
```

To exit:

Press:

```
q
```

12. Creating GitHub Account

Go to:

[GitHub Official Website](https://github.com)

Create account using:

- Name
- Email
- Password

13. Creating Repository on GitHub

Steps

1. Login to GitHub
2. Click **New Repository**
3. Enter repository name

Example:

portfolio-website

4. Click **Create Repository**

14. Connect Local Project to GitHub

Step 1: Copy Repository URL

Example:

`https://github.com/username/project.git`

Step 2: Connect Repository

Command:

```
git remote add origin repository-url
```

Example:

```
git remote add origin https://github.com/aqib/portfolio.git
```

Meaning

Connects local project to GitHub repository.

15. Push Code to GitHub

First Push Command

```
git branch -M main
```

Makes main branch.

Push Command

```
git push -u origin main
```

Meaning

Uploads project to GitHub.

After this:

Refresh GitHub → Files appear online.

16. Clone Repository

What is Clone?

Copying a GitHub project to your computer.

Command

```
git clone repository-url
```

Example:

```
git clone https://github.com/user/project.git
```

Git downloads full project.

17. Pull Command

Purpose

Gets latest changes from GitHub.

Command

```
git pull origin main
```

Example Situation

Friend updated code → You run:

```
git pull origin main
```

Now latest code comes into your system.

18. Basic Branching

What is Branch?

A separate version of project.

Used to test new features without affecting main project.

Example:

Main website running.

You create another branch for navbar testing.

Create Branch

```
git branch navbar
```

Check Branches

```
git branch
```

Output:

```
* main
navbar
```

* means current branch.

Switch Branch

```
git checkout navbar
```

Now moved to navbar branch.

Merge Branch

Go to main branch:

```
git checkout main
```

Merge:

```
git merge navbar
```

Now navbar changes added to main.

19. Merge Conflict

What is Merge Conflict?

Happens when same file line is changed in two branches.

Example:

Branch A:

```
<h1>Hello</h1>
```

Branch B:

```
<h1>Welcome</h1>
```

Git gets confused.

You manually choose correct code.

20. .gitignore File

Purpose

Ignore files/folders.

Git will not track them.

Example:

```
node_modules
.env
```

Create File

```
.gitignore
```

Inside:

```
node_modules
.env
```

Why?

Some files are very large or private.

21. Full Practical Flow

Step 1

Create project folder.

Example:

```
portfolio
```

Step 2

Initialize Git

```
git init
```

Step 3

Check status

```
git status
```

Step 4

Add files

```
git add .
```

Step 5

Commit changes

```
git commit -m "Initial project setup"
```

Step 6

Create repository on GitHub

Example:

```
portfolio-project
```

Step 7

Connect GitHub

```
git remote add origin https://github.com/username/portfolio.git
```

Step 8

Push code

```
git branch -M main  
git push -u origin main
```

Project uploaded successfully.

22. Important Commands Summary

Command	Purpose
<code>git init</code>	Initialize Git
<code>git status</code>	Check status
<code>git add .</code>	Add files
<code>git commit -m ""</code>	Save changes
<code>git log</code>	Show history
<code>git clone</code>	Download repository
<code>git pull</code>	Get latest code
<code>git push</code>	Upload code
<code>git branch</code>	Create branch
<code>git checkout</code>	Switch branch
<code>git merge</code>	Merge branch